A Formal MDE Framework for Inter-DSL Collaboration

Salim Chehida(✉), Akram Idani, Mario Cortes-Cornax, and German Vega

Univ. Grenoble Alpes, CNRS, Grenoble INP, LIG, 38000 Grenoble, France
{salim.chehida,akram.idani,mario.cortes-cornax,
german.vega}@univ-grenoble-alpes.fr

Abstract. In order to master the complexity of a system at the design stage, several models have to be defined and combined together. However, when heterogeneous and independent DSLs are used to define these models, there is a need to explicitly compose their semantics. While the composition of static semantics of DSLs is straightforward, the coordination of their execution semantics is still challenging. This issue is generally called inter-DSL collaboration. In this paper, we propose a formal Model Driven Engineering (MDE) framework built on the Meeduse language workbench that we extend with the Business Process Model and Notation (BPMN). Meeduse allows to instrument DSLs with formal semantics using the B method. BPMN provides an easy-to-use notation to define the coordination of execution semantics of these DSLs. A transformation of BPMN models into Communication Sequential Process (CSP) formal language enables the possibility for animation and verification. Our approach is successfully demonstrated by modeling the collaboration of two DSLs from a real case study.

Keywords: DSL · BPMN · Model Composition · Models Collaboration · Formal Methods · B Method · CSP · Animation · Verification

1 Introduction

Domain-Specific Languages (DSLs) can be used to model different concerns of a system such as its architecture, the inherent processes, data flows or security policies. These dedicated languages permit domain experts to create models in their speciality. However, models that are built with heterogeneous DSLs must be combined together in order to favour maintenance, verification, code generation, etc. Indeed, updating a model usually leads to modifications of the other related models. Also, triggering an action in one model can induce other actions in the related models. This situation has been observed in real applications, leading to the necessity to bridge the gap between DSLs semantics and preventing inconsistencies all along a model-driven development process [12].

In this paper, we propose a novel approach, supported with a formal framework, that allows engineers to define how DSLs collaborate with each other.

S.-S. Jongmans and A. Lopes (Eds.): COORDINATION 2023, LNCS 13908, pp. 232–249, 2023.
https://doi.org/10.1007/978-3-031-35361-1_13

We use Meeduse [10], the only existing language workbench (LWB) today that enables both formal reasoning (via theorem proving) and the DSL execution (via animation and model-checking). Meeduse uses the B method [1] for specifying the semantics of DSLs and ProB model checker [18] for animation and verification. It has been successfully applied to several realistic case studies [11].

However, the tool did not present any feature to deal with DSL collaboration. Hence, the goal of this work is to extend Meeduse with the capability to define, execute and verify inter-DSL collaboration. Our approach consists of three main steps:

1. In the first step, a composition metamodel is created in order to relate the semantic domains of independent DSLs, assuming that these semantics are themselves defined by means of metamodels;
2. In the second step, Meeduse is used to transform the various metamodels into formal B specifications taking benefit of the composition mechanism of the B method. The resulting specifications are then used to define the execution semantics of these metamodels via B operations;
3. The third step uses BPMN [20] to define the collaboration between the DSLs. In this model, tasks refer to B operations providing the execution workflow of the collaboration. By transforming the BPMN model into CSP (Communication Sequential Process [22]), we obtain a CSP||B [23] specification that is used for animating and verifying the so-called DSLs' collaboration.

To illustrate our approach, we apply it to a smart grid case study provided by RTE, the energy transmission company in France. The case study involves two DSLs: the first one focuses on the management of system configurations assigning to a set of applications various infrastructures. The second is dedicated to security risk assessment. The composition and the collaboration of these DSLs allow to manage configurations while dealing with security concerns.

Following the introduction, we describe a motivation scenario in Sect. 2. We present our model-based approach for inter-DSL collaboration in Sect. 3. Then, the application of our approach to the RTE case study is discussed in Sect. 4. Section 5 lists related works and finally, Sect. 6 draws the conclusions and the perspectives of this paper.

2 Case Study and Motivation

In this work, we consider a real case study inspired from [24]. To promote renewable energies, RTE aims to make extensive use of smart grid technologies and develop efficient systems that respond to *meteorological* and *security* factors. The main challenge is to deal with overcurrent in the transmission lines due to the excess energy production in the wind farms, which can create a real danger for people and goods near the transmission lines. For this purpose, a first DSL, named *CM-DSL* (Configuration Management DSL), is used for defining the system configurations to be used for responding to the environment dynamics. A second DSL, named *SRA-DSL* (Security Risk Assessment DSL), is used for security modeling and analysis. In the following sections, more details about the aforementioned DSLs are given.

2.1 Configuration Management DSL (CM-DSL)

The Configuration Management DSL (CM-DSL) describes RTE configurations by assigning applications to infrastructures. In this work, we present a simplified part of CM-DSL to illustrate the problem. Figure 1 presents a model, where three applications are defined: (1) *Fast Action* uses a simple flow chart logic and leads to the massive disconnection of wind farms from the grid; (2) *Normal* refers to predictive control algorithm in order to seek the optimal use of all levers (wind farms modulation, batteries and circuit breakers); and finally, (3) *Enhanced Forecasting* uses forecasts of the next day generation and weather (wind and sunlight) for optimisation. The model also considers three infrastructures: (1) in *Centralized RTE*, algorithms can only be run on data-center resources that communicate directly with the gateways connected to sensors or actuators for generators or batteries; (2) in *Collaborative Fog*, the execution is done in substation calculators that communicate in a peer-to-peer scheme with the other calculators; and (3) in *Hierarchical Fog-Cloud*, all calculators are available to run the applications and both data-center to substation and substation to substation communications are allowed. Combining applications and architectures a configuration can be proposed. Figure 1 presents four different ones. A configuration is also characterized by its response time (QoS in seconds) calculated using an external simulation tool. The status is decided by the software architect, which can be valid (example of AM1-IM3) or invalid (example of AM3-IM1) depending on its QoS. In the selected *AM1-IM2* configuration (framed in dotted line), the *Fast Action* application is to be deployed on *Collaborative Fog* infrastructure. Valid configurations can be used in the real operation.

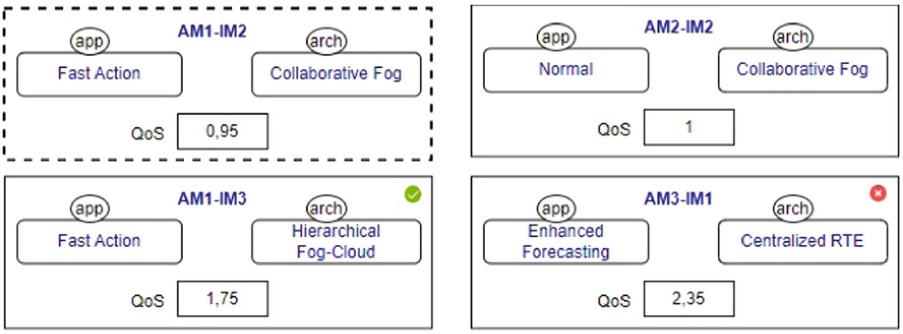

Fig. 1. A Configuration Management (CM) model showing four different configurations

2.2 Security Risk Assessment DSL (SRA-DSL)

The Security Risk Assessment DSL (SRA-DSL) refers to Attack-Defense Tree [15] for security risk assessment. A tree structure is used for specifying threats,

defenses, and the combinations between them. In Fig. 2, threat nodes are represented by rectangles and defense nodes by parallelograms. The combinations between the nodes are expressed by logical operators (AND, OR, NOT) depicted by ellipses. The SRA-DSL model of Fig. 2 specifies the combination between the 8 threats (T1,T2, ...,T8) and the 5 defenses (D1,D2, ...,D5) of the RTE system. Each defense prevents a set of threats. For instance, *closing a specific breach in the network* (D2) can prevent *tampering on customer network* (T5), *denial of service* (T6), and *information disclosure* (T7). In a SRA-DSL model, it is possible to select the possible threats (framed in bold as for instance T8) on a part of the system or in a specific state of the system among the whole set of system threats. It is also possible to compute the defenses subset (framed in blue bold as for instance D4 and D5) that can counter all the selected threats. The SRA-DSL user can also set the defense *Cost* in seconds, which is the delay caused by its activation to protect the system (an example cost of D1 is 0.4 s).

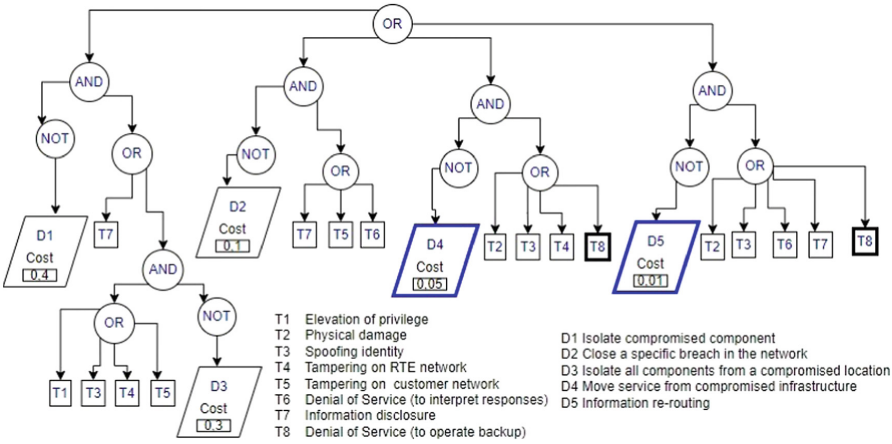

Fig. 2. The SRA model

We note that CM-DSL and SRA-DSL are independent and can be used for other applications or case studies. For example, paper [4] shows how the SRA-DSL is used for security risk assessment in Internet of Things (IoT) systems.

2.3 Collaboration and Verification Needs

CM-DSL provides the possible configurations that can meet the environmental dynamics, but it does not consider the configurations' security. On the other side, the SRA-DSL is a logic-based DSL that can be used within different security contexts. In our case study, we use SRA-DSL to identify possible threats of each configuration and to calculate the required defenses. By doing so, the response time and the validation state of the configuration can be updated based on the

cost of the defenses. The integration of these two DSLs infers the need to ensure inter-DSL properties. Below, we provide some examples of properties:

- (P1) The delay caused by configuration defenses should not exceed a percentage of the configuration response time (e.g., 30%)
- (P2) At least one defense must be activated to secure a configuration.
- (P3) The response time of a configuration should not exceed a given threshold (e.g., 2 s).

In this work, we propose a new approach to manage and check the collaboration between two or more DSLs (in this case, illustrated by the CM-DSL and the SRA-DSL). We aim more precisely to:

- Highlight the links between the DSLs concepts (e.g., configuration and defenses).
- Specify the collaboration process between the DSLs actions.
- Animate interactively the collaboration process while ensuring the updates at the DSL models.
- Express and check inter-DSL properties.

In the next section, we describe the Inter-DSL approach that orchestrates the DSLs' execution collaboration.

3 Inter-DSL Collaboration: A Model-based Architecture

An overview of our approach is given in Fig. 3, divided into two main blocks: X and Y. The model-based approach is supported by the Meeduse platform for rigorous DSLs' design (block X presented in Sect. 3.1). Meeduse has been extended for modeling, animation and verification of inter-DSL collaboration (block Y presented in Sect. 3.2). Figure 3 shows the collaboration between two DSLs, but the approach can be applied for more.

3.1 Formal Model Driven DSLs

As shown in Fig. 3, we first specify the abstract syntax of each DSL using EMF-based modeling tool (Ecore, Xtext, Sirius, GMF, etc.). The DSL's concepts are represented by a metamodel. The different metaclasses are characterized by a set of attributes and operations, related by a set of associations (see examples in Fig. 4). The domain expert uses the DSL to create models (instances) that conform to the DSL metamodel.

We instrument our DSLs in Meeduse language workbench, which provides a formal framework to our approach. Meeduse produces a B machine from a given metamodel and enables the application of B method to define the execution semantics of the DSLs together with its invariant properties. The static semantics of the DSL is represented by sets, variables and typing invariants that define the structural features of the metamodel. The execution semantics of the

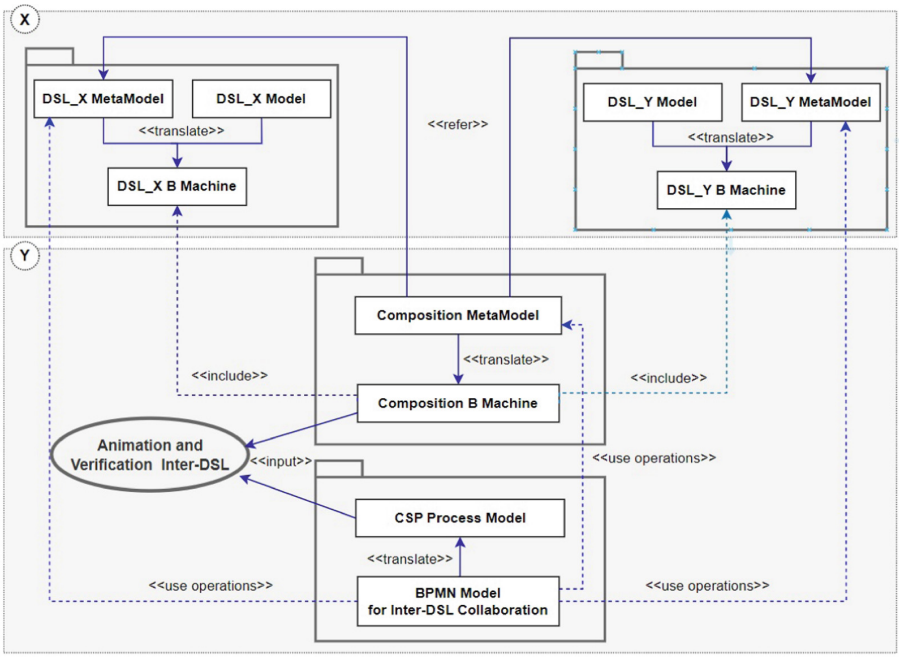

Fig. 3. Generic architecture for modeling and verification of inter-DSL collaboration

DSL is represented by operations that define the DSL actions and initialization. Meeduse is also equipped with a tool that allows to animate the execution of a model (instance of the metamodel) into the DSL B machine, by introducing the valuation of the B abstract sets and the initialization of B variables.

As mentioned earlier, the Meeduse uses ProB for the animation and verification of B specifications. In this step, each DSL can be animated separately by running its actions represented by B operations. At each animation step, the animator provides the set of operations that could be executed, which are the ones that preserve the invariants. In this work, we aim to collaboratively animate a set of DSLs by triggering actions from different DSLs while respecting the collaboration process and system properties.

3.2 Formal Model Driven Inter-DSL Collaboration

The inter-DSL collaboration is specified by a so-called "composition metamodel" and a BPMN collaboration diagram, which will then be transformed into formal specifications to be used for animation and verification.

Composition. The composition metamodel links the other DSL metamodels. It is represented by a metaclass named Composition related by composition associations to the root metaclasses of each DSL to be coordinated. In this metamodel,

we can also add new metaclasses with new attributes, operations (composition actions) and references to meet inter-DSL collaboration needs. Starting from this metamodel, we generate the composition B machine that includes the B specifications obtained from the different DSLs. In the dynamic part of the composition B machine, we integrate the B operations defining actions involved in the collaboration process. We can also introduce properties like those presented in the Sect. 2.3 as invariants of the composition B machine.

Collaboration Model. We express the collaboration between the DSLs actions using a process modeling language. For process modeling, there are several languages and notations, such as the Business Process Model and Notation (BPMN) and the Unified Modeling Language (UML) Activity Diagram [21]. In this work, we use BPMN 2.0, which is the standard de-facto for process modeling. BPMN, supported by the Object Management Group (OMG), is a language that proposes a graphical notation providing the description of processes elements supported by a metamodel, without specifying a fully defined execution semantics (in natural language). In our approach, we use the notion of BPMN pool, which is the graphical representation of a participant in a collaboration, to group operations of each DSL, including the composition metamodel. We represent an atomic action specifying one metamodel operation by a BPMN task and we use expanded subprocesses to represent a grouping of tasks. BPMN sequence flows are used to represent the sequence of actions in the context of a DSL (inside the Pool), while message flows are used to represent the inter-pool communication. Gateways (exclusive or parallel) model the control flow in each DSL.

Formal Process Model. To enable the animation and verification of inter-DSL collaboration, the graphical BPMN models are mapped to a formal specification. Several works propose this transformation to process algebras languages like CSP or Calculus of Communicating Systems (CCS) [19]. In this paper, we manually transform the BPMN diagrams into CSP models. Work in progress intends to automate and validate this transformation by exploiting works like [8]. The constructor's mapping is illustrated with our use case in Sect. 4.3. From this mapping, we aim to take advantage of the ProB tool (integrated into the Meeduse platform) for the animation and verification. The ProB tool takes as input the CSP process model and the composition B machine coming from the DSLs and composition metamodels respectively. The CSP||B specification, as presented in [3], is used for animating and verifying the inter-DSL collaboration processes.

4 Application to Smart Grid System

In our approach, the *Model-driven engineer* specifies the DSLs metamodels and the BPMN models of their collaboration. Then, the metamodels and BPMN diagrams are transformed into B and CSP respectively, while integrating the system

properties. Afterwards, the *operator* can animate the formal specifications while observing the respect of the properties.

In [5], we describe the different steps of our approach and its application to our case study. We also provide the different artifacts of this case study. The modeling part includes the CM-DSL and SRA-DSL metamodels, the metamodel of their composition, the models that define the instances of the metamodels, and the BPMN diagram that describes the collaboration between the DSLs. The formal specification part includes the B machines generated from the DSLs and their composition as well as the CSP model specified from the BPMN diagram.

4.1 Modeling DSLs and Their Collaboration

Metamodels. Figure 4 shows the metamodels that we propose for our case study (one per DSL, and a third one for the composition). Part A of Fig. 4 is the EMF metamodel of CM-DSL. The root class (CM) is composed of a set of applications and a set of infrastructures. Every application applies a specific optimisation algorithm that gets the production parameters and information from sensors to trigger actions such as limiting the production on wind farms or charging batteries. Infrastructures are based in RTE power stations and datacenters, connected by a private telecommunication network. A CM model is also composed of configurations, each one consists of the combination of one application and one infrastructure and it is characterized by its response time (QoS) and validation state (isValid). The CM-DSL defines the following operations : *selectConfig* selects one configuration from the list of CM configurations, *validateConfig* validates the selected configuration, and *setQoS* introduces or modifies the QoS of the selected configuration.

In part B of Fig. 4 the SRA-DSL metamodel is shown, which is composed of metaclasses for specifying the possible threats of the system and the defenses that can be deployed to protect the system. Each defense is characterised by a cost (costDef), which is the delay caused by its activation in the RTE platform. A series of metaclasses are introduced for representing the logical operators (NOT, OR, AND) combining threats and defenses. The operations of SRA-DSL are : *initSRA* selects the subset of possible threats, *selectThreat* selects a threat from the set of possible threats, *computeDefenses* calculates the subset of defenses that can block the selected threats.

Finally, part C of Fig. 4 shows the composition metamodel. Metaclass COMPOSITION is composed of the root metaclasses of the previously presented DSLs (CM-DSL and SRA-DSL). The composition has its own operations that define its execution semantics. Indeed, we introduce the notion of *Secure Configuration* that associates a set of defenses to a valid configuration (using operation *affectValidDefenses*). Furthermore, operation *approveSecureConfig* allows one to validate a Secure Configuration if it ensures risk assessment and quality of service properties.

Inter-DSL Collaboration. In our proposal, we describe the inter-DSL collaboration using a BPMN model (Fig. 5) relying on the operations of the above

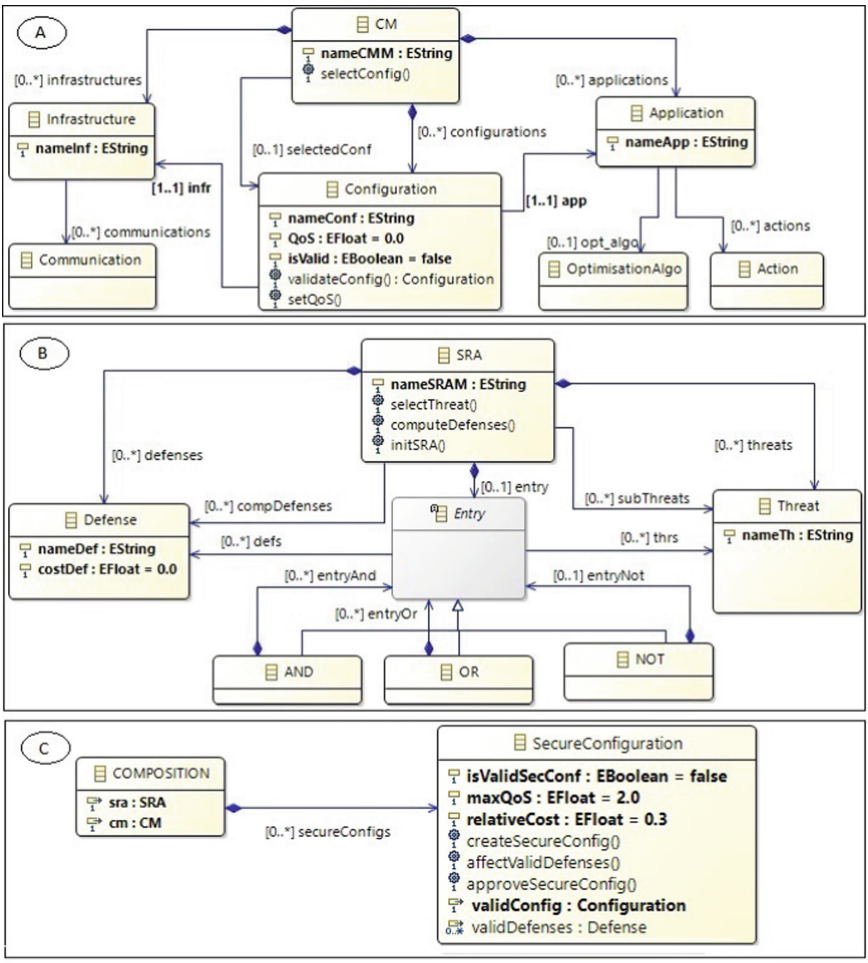

Fig. 4. DSL and composition metamodels

metamodels. We use the notion of Pool to separate the execution semantics of every DSL. The collaboration starts in pool DSLs_COMPOSITION, where it triggers via a message (named *ConfigurationRequest*) the selection and validation of a configuration at the CM-DSL level. Note that this corresponds to a functional validation, not considering yet the security aspects. Afterwards, in the composition pool, a new secure configuration is created and associated to the validated configuration. This task is triggered by the reception of message *ValidConfig*. The new secure configuration is then sent to the SRA-DSL pool in order to select possible threats of this configuration and compute the defenses that can prevent them. When receiving the defenses (message *ValidDefenses*), it is possible to approve the secure configuration after checking the global QoS and the other risk assessment properties.

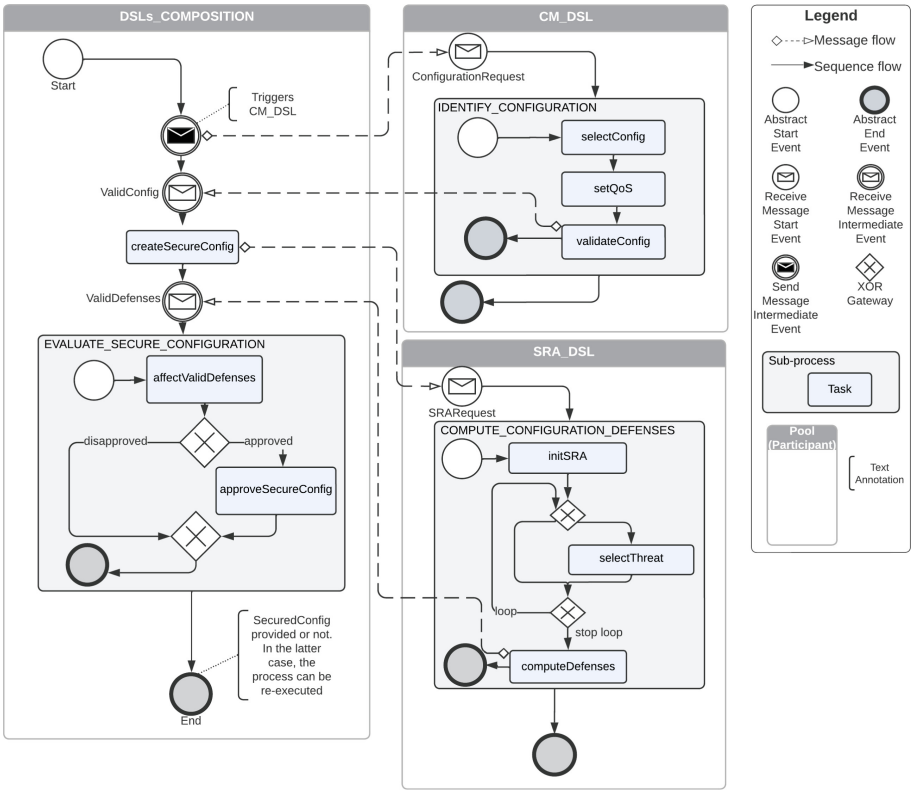

Fig. 5. BPMN collaboration model of SRA-DSL and CM-DSL

4.2 Formalization of Metamodels

While metamodels describe static semantics of DSLs, the BPMN model and the associated operations represent their execution semantics. We need on the one hand a tool to define and animate these semantics and on the other hand a way to guarantee that the security properties are preserved by the underlying behaviour. To this purpose the Meeduse framework has been extended. The tool applies the B method to formally define execution semantics of DSLs using B operations, which allows us to introduce the security properties using B invariants. From this formal specification the AtelierB prover is applied in order to guarantee the correctness of the model. In this section, we start by presenting an excerpt of the resulting B specifications, and then we discuss the CSP model that is built from our BPMN diagram.

Given a metamodel, Meeduse generates a formal B specification in which classes are defined using sets. Attributes and associations are defined using functional relations. Not all of our specifications are presented but only an example, as we rather focus on the composition metamodel. The header part of the corresponding specification is given below. This B machine includes machines

CM_DSL and *SRA_DSL* issued from our DSLs and where operations *selectConfig*, *validateConfig*, *setQoS*, *selectThreat*, *computeDefenses* and *initSRA* are defined.

MACHINE *DSL_Composition*
INCLUDES *CM_DSL*, *SRA_DSL*
PROMOTES
selectConfig, *validateConfig*, *setQoS*, `/* from CM DSL */`
selectThreat, *computeDefenses*, *initSRA* `/* from SRA DSL */`

Figure 6 gives the B data structures and their typing invariants that are produced by Meeduse from the composition metamodel. Specializations of functional relations depend on the character of attributes and associations: single/multiple valuated and mandatory/optional. Class *SecureConfiguration* is transformed into an abstract set named *SECURECONFIGURATION* that represents the set of possible instances of the class. Variable *SecureConfiguration* defines the effective instances of this class. We introduced in this machine the variable *theSecConf* in order to represent the current secure configuration that is managed by a given process.

SETS
COMPOSITION; *SECURECONFIGURATION*
VARIABLES
SecureConfiguration, *sra*, *cm*, *validConfig*, *validDefenses*, *secureConfigs*, *isValidSecConf*, *maxQoS*, *relativeCost*, *theSecConf*
INVARIANT
`/* Object Definition */`
$SecureConfiguration \subseteq SECURECONFIGURATION \wedge$
`/* Associations */`
$sra \in COMPOSITION \rightarrow SRA \wedge$
$cm \in COMPOSITION \rightarrow CM \wedge$
$validConfig \in SecureConfiguration \rightarrow Configuration \wedge$
$validDefenses \in SecureConfiguration \leftrightarrow Defense \wedge$
$secureConfigs \in SecureConfiguration \nrightarrow COMPOSITION \wedge$
`/* Attributes */`
$isValidSecConf \in SecureConfiguration \rightarrow \mathbf{BOOL} \wedge$
$maxQoS \in SecureConfiguration \rightarrow REAL \wedge$
$relativeCost \in SecureConfiguration \rightarrow REAL \wedge$
`/* added variable */`
$theSecConf \in SECURECONFIGURATION$

Fig. 6. B data of the composition machine

Regarding the inter-DSL properties P1, P2 and P3 introduced in Sect. 2, they are formally defined as follows. This invariant means that if a secure configuration is created (*i.e.* becomes an existing instance) and validated, then P1, P2 and P3 hold.

DEFINITIONS
$sumDefCost(sc) == \sum def \,.\, (def \in validDefenses[\{sc\}] \mid costDef(def))$
$approve(sc) ==$
$sumDefCost(sc) + QoS(validConfig(sc)) \leq maxQoS(sc)$
$\wedge\ sumDefCost(sc) \leq relativeCost(sc) \times QoS(validConfig(sc))$
$\wedge\ validDefenses[\{sc\}] \neq \emptyset$
INVARIANT
$theSecConf \in SecureConfiguration \wedge isValidSecConf(theSecConf) = \textbf{TRUE}$
$\Rightarrow approve(theSecConf)$

The dynamic parts of the various B machines contain B operations that specify the execution semantics of our DSLs. We provide for example the specification of operation *approveSecureConfig* that is defined in the *DSL_Composition* machine:

OPERATIONS
approveSecureConfig =
PRE
$theSecConf \in SecureConfiguration$
$\wedge\ isValidSecConf(theSecConf) = \textbf{FALSE}$
$\wedge\ approve(theSecConf)$
THEN
isValidSecConf$(theSecConf) := \textbf{TRUE}$
END

All our specifications have been proved correct via the AtelierB prover [6]. The latter produces Proof Obligations (POs), which correspond to theorems that require proof in order to verify the correctness and consistency of a B machine. AtelierB offers an automatic prover able to prove the majority of POs automatically, without any user interaction. This makes it easy to quickly verify large parts of the system specification, and reduces the burden on the user to manually prove each individual PO. The prover also provides interactive prover designed for handling and finalizing complex proofs, also for detecting errors in the specifications.

With AtelierB, we generated 299 POs. Most of them have been proved automatically; only 10 POs have been discharged manually using the interactive prover. The latter ones required additional information to be completed and corrected. The proof using AtelierB provides the guarantee that all the invariants of our DSLs are preserved by the formal specifications of their execution semantics and that the composition is correct with regards to these invariants and the security properties.

4.3 BPMN Formalization with a CSP Transformation

Having the B specifications, the formalization of the coordination model can be treated. Our objective is to apply the workflow of the BPMN model to the B operations issued from the machine *DSL_Composition*. To this purpose, we extend Meeduse with a transformation from BPMN into CSP, which allows us

to apply the approach of CSP||B [23]. This technique is inspired by the work of M. Kleine [13] about the usage of CSP as a coordination language. The idea is to coordinate (non-)atomic actions (in our case these are B operations) of a system by a CSP-based coordination environment in a noninvasive way, meaning that actions do not need to be modified to be coordinated. In this sense, the formal B specifications that define the semantics of our DSLs remain unchanged. We just layer a CSP model on top of them, to distinguish the coordination concerns from state related properties.

In CSP, a process represents an independent entity that performs a sequence of events, which is similar to the notion of pool in BPMN. Communication between processes is ensured via channels, that may or not transmit data flows. We use this notion to represent exchanged messages represented in the BPMN model.

Accordingly, to transform the BPMN collaboration model we first transform pools leading to independent CSP processes (Fig. 7), and then we produce a main process (Fig. 8) to synchronize them being guided by the message exchanges between pools. Note that by convention, processes are named in uppercase and channels in lowercase. The used CSP constructs are:

```
Process ::= SKIP                 /* terminating process */
          | ch -> Process        /* simple action prefix where ch is a channel */
          | Process ; Process    /* sequential composition */
          | Process [] Process   /* external choice */
```

A channel `ch` may transmit data `d`, which is denoted as `ch?d` for reading data, and `ch!d` for writing data. Furthermore, a process can be defined with parameters such as `Process(param`$_1$`,...,param`$_n$`)`. Inputs, outputs and process parameters are useful in our case since the communication between pools is done via messages that may contain some data. For example, message `ConfigurationRequest` of `CM_DSL` enables process `IDENTIFY_CONFIGURATION` without any data; on the other hand, message `ValidConfig` is produced by `validateConfig` and received by `createSecureConfig`, and contains a valid configuration. This data is represented with parameter `conf`. In one case it is an input data and in the other case it is an output data.

The synchronisation of processes `CM_DSL`, `SRA_DSL` and `DSLs_COMPOSITION` is done in the `MAIN` process (see Fig. 8). This process applies a parallel composition with channels' synchronisation, representing the exchanged messages. For instance, processes `CM_DSL` and `DSLs_COMPOSITION` are synchronised on channels `ConfigurationRequest` and `ValidConfig`. As their starting point is channel `ConfigurationRequest` they are both triggered at the same time. However, the next step of `DSLs_COMPOSITION` is `ValidConfig?conf`, which means that `DSLs_COMPOSITION` is blocked until `CM_DSL` reaches `ValidConfig!conf`. This execution conforms to the BPMN model since the coordination starts by asking CM_DSL to provide a valid configuration and next, a secure configuration is created from this valid configuration.

```
/* Transformation of CM_DSL Pool */
1.  CM_DSL =
2.      ConfigurationRequest → IDENTIFY_CONFIGURATION ; CM_DSL
3.  IDENTIFY_CONFIGURATION =
4.      selectConfig → setQoS → validateConfig?conf → ValidConfig!conf → SKIP
```

```
/* Transformation of SRA_DSL Pool */
5.  SRA_DSL =
6.      SRARequest → COMPUTE_CONFIGURATION_DEFENSES ; SRA_DSL
7.  COMPUTE_CONFIGURATION_DEFENSES =
8.      initSRA → SELECT_THREATS
9.      ; computeDefenses?defenses → ValidDefenses!defenses → SKIP
10. SELECT_THREATS =
11.     selectThreat → SELECT_THREATS [] SKIP
```

```
/* Transformation of DSLs_COMPOSITION Pool */
12. DSLs_COMPOSITION =
13.     ConfigurationRequest → ValidConfig?conf → createSecureConfig!conf
14.     → SRARequest → ValidDefenses?defenses
15.     → EVALUATE_SECURE_CONFIGURATION (defenses)
16.     ; DSL_COMPOSITION
17. EVALUATE_SECURE_CONFIGURATION(defenses) =
18.     affectValidDefenses!defenses → (approveSecureConfig → SKIP [] SKIP)
```

Fig. 7. Transformation of pools into CSP

```
/* Main process */
MAIN =
    CM_DSL
        [|{|ConfigurationRequest, ValidConfig|}|]
    DSLs_COMPOSITION
        [|{| SRARequest, ValidDefenses|}|]
    SRA_DSL
```

Fig. 8. Parallel synchronisation of pools

4.4 Discussion

The CSP model and the B specifications that describe the semantics of DSLs provide a formal framework that is managed by ProB and Meeduse in order to ensure the animation as well as the verification. First, Meeduse valuates the formal B specifications from the models of Figs. 1 and 2. The resulting valuations are then delivered to ProB, which allows a step-by-step animation of B operations. The execution of DSLs in Meeduse using ProB is well mastered today and has been discussed in [10]. However, the shortcoming of this approach is the absence of guidance during animation, which makes it inconvenient for DSL collaboration. In order to address this limitation, we suggest to apply the CSP||B approach. The idea is to marry CSP and B such that the execution of a

B operation corresponds to an event that can be enabled in CSP, which provides a guidance all along the animation process. Roughly speaking, the B machine and the CSP process must synchronise on common events, that is, an operation can only happen in the combined system when it is allowed both by the B and the CSP. For more details about this technique we refer the reader to [3].

We have tested several executions of our coordination model based on our case study, leading to the creation of several secure configurations with various threats and defenses. Figure 9 shows the animation of the composition B machine presented in Sect. 4.2 guided by the CSP model of Figs. 7 and 8, using the ProB tool. In the *History* window, an example of execution scenario that creates a new secure configuration is shown. The *Enabled Operations* window lists operations that can be called at this stage and whose execution will satisfy their precondition, and therefore preserve the state invariant. The *State Properties* window provides the current value of each state variable of the composition machine. At the animation step, we can observe the insurance of system properties P1, P2 and P3 introduced in Sect. 2.3 and specified as invariants in the composition B machine in Sect. 4.2.

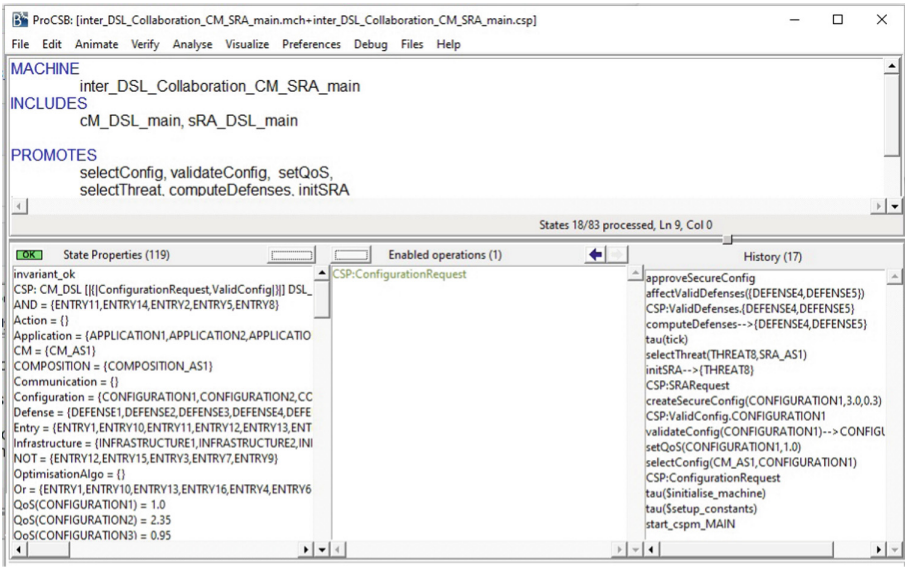

Fig. 9. ProB screenshot showing animation and verification of inter-DSL collaboration

Regarding verification, one of the advantages of B is the availability of theorem provers and model-checkers. We did not discuss in details this activity but note that all our formal specifications have been proved, which guarantees the correctness of the B models on which we built the coordination process. Indeed, our approach ensures the correctness and reliability of the resulting RTE system,

which can help preventing dangerous situations that can be caused by overcurrent in the transmission lines.

5 Related Work

The problem of managing the interactions between models is not new and it has been addressed in various fields including Artificial Intelligence, Robotics, and Control Systems. In the context of Model Driven Engineering (MDE), the proposed solutions are categorized into two classes: composition and coordination approaches [16]. The composition approaches build a model specifying the structural relationships between model elements. The composition model is specified using matching operators that define the syntactic similarities between model elements and merging operators that implement rules to specify how the matched elements must be composed. The operators are provided by dedicated languages such as Epsilon [14]. The composition model can also be defined by composing the syntax of different model languages into a new language syntax expressed by a metamodel [7]. As part of the composition approaches, F. Jouault et al. [12] combine megamodeling and model weaving techniques to build an environment that ensures traceability and navigation between the DSLs models. The environment represents links between different DSL models, which provides an intuitive way to understand relationships between them. However, these composition approaches consider only the language's static syntax; they focus on traceability without taking into account execution semantics.

In coordination approaches, a new model is often built to define how the models behave and interact with each other. Dedicated languages (*e.g.* Linda [9]) have been proposed for this purpose. There also exist some coordination frameworks, such as ModHel'X [2], to automate and support the coordination process. However, the coordination is encoded using a general-purpose programming language (Java), which is not suitable for performing verification and validation. As part of the GEMOC project[2], Larsen et al. [17] propose the B-COoL language that allows specifying a coordination pattern between DSLs. The language offers concurrency and communication models representing the control-flow aspects, including the synchronisation and the causality relationships between execution functions. This is close to some CSP operators, but the missing piece is the usage of provers and model-checkers for automatic verification activities. In our work we apply well-known formal languages (B and CSP) and an established semi-formal notation (BPMN), which enables the application of widely used tools for modeling and verification.

Our approach combines the composition and coordination techniques. We build a composition metamodel that refers to the coordinated DSLs metamodels without altering their semantics and behaviors. BPMN diagram is used for expressing the coordination process between the DSLs. The DSLs metamodels and the BPMN coordination model are then transformed into a formal specification to be used for checking and validating the DSLs models coordination.

[2] https://gemoc.org/

6 Conclusion

This paper presented a tool-supported approach for the formal modeling and verification of inter-DSL collaboration. The approach extends the Meeduse framework by integrating a composition metamodel and BPMN model to explicitly specify inter-DSL collaboration. The formal specification generated from the models and metamodels is then used to verify the DSLs models coordination. The proposed approach was demonstrated to be effective through its successful application in a real case study.

Going forwards, the next steps include implementing and validating the transformation rules from BPMN to CSP, as well as expressing and checking properties at the workflow level. These developments will further enhance the automation and reliability of the proposed approach.

Data Availability Statement

The artifact is available in the Software Heritage repository:

swh:1:dir:c38d7336f13f438eab7212227f10fb0dbf0350c1

References

1. Abrial, J.R.: The B-book: assigning programs to meanings. Cambridge University Press (1996). https://doi.org/10.1017/CBO9780511624162
2. Boulanger, F., Hardebolle, C.: Simulation of multi-formalism models with Modhel'x. In: 2008 1st International Conference on Software Testing, Verification, and Validation, pp. 318–327 (05 2008). https://doi.org/10.1109/ICST.2008.15
3. Butler, M., Leuschel, M.: Combining CSP and B for specification and property verification. In: Fitzgerald, J., Hayes, I.J., Tarlecki, A. (eds.) FM 2005. LNCS, vol. 3582, pp. 221–236. Springer, Heidelberg (2005). https://doi.org/10.1007/11526841_16
4. Chehida, S., Baouya, A., Bozga, M., Bensalem, S.: Exploration of impactful countermeasures on IoT attacks. In: 9th Mediterranean Conference on Embedded Computing, MECO 2020, Budva, Montenegro, 8–11 June 2020, pp. 1–4. IEEE (2020). https://doi.org/10.1109/MECO49872.2020.9134200
5. Chehida, S., Idani, A., Cortes-Cornax, M., Vega, G.: GitHub artifacts. http://github.com/SalimChehida/Inter-DSL-Collaboration
6. Clearsy: Atelier B. http://www.atelierb.eu/en/
7. Emerson, M., Sztipanovits, J.: Techniques for metamodel composition. In: OOPSLA - 6th Workshop on Domain Specific Modeling (2006)
8. Flavio, C., Alberto, P., Barbara, R., Damiano, F.: An ECLIPSE Plug-in for formal verification of BPMN processes. In: 2010 Third International Conference on Communication Theory, Reliability, and Quality of Service, pp. 144–149 (2010). https://doi.org/10.1109/CTRQ.2010.32
9. Gelernter, D., Carriero, N.: Coordination languages and their significance. Commun. ACM **35**(2), 97–107 (1992)

10. Idani, A.: Meeduse: a tool to build and run proved DSLs. In: Dongol, B., Troubitsyna, E. (eds.) IFM 2020. LNCS, vol. 12546, pp. 349–367. Springer, Cham (2020). https://doi.org/10.1007/978-3-030-63461-2_19
11. Idani, A., Ledru, Y., Ait Wakrime, A., Ben Ayed, R., Bon, P.: Towards a tool-based domain specific approach for railway systems modeling and validation. In: Collart-Dutilleul, S., Lecomte, T., Romanovsky, A. (eds.) RSSRail 2019. LNCS, vol. 11495, pp. 23–40. Springer, Cham (2019). https://doi.org/10.1007/978-3-030-18744-6_2
12. Jouault, F., Vanhooff, B., Bruneliere, H., Doux, G., Berbers, Y., Bezivin, J.: Inter-DSL coordination support by combining megamodeling and model weaving. In: Proceedings of the 2010 ACM Symposium on Applied Computing, pp. 2011–2018. SAC 2010, Association for Computing Machinery, New York, NY, USA (2010)
13. Kleine, M.: CSP as a coordination language. In: De Meuter, W., Roman, G.-C. (eds.) COORDINATION 2011. LNCS, vol. 6721, pp. 65–79. Springer, Heidelberg (2011). https://doi.org/10.1007/978-3-642-21464-6_5
14. Kolovos, D.S., Paige, R.F., Polack, F.A.C.: Merging models with the epsilon merging language (EML). In: Nierstrasz, O., Whittle, J., Harel, D., Reggio, G. (eds.) MODELS 2006. LNCS, vol. 4199, pp. 215–229. Springer, Heidelberg (2006). https://doi.org/10.1007/11880240_16
15. Kordy, B., Mauw, S., Radomirović, S., Schweitzer, P.: Foundations of attack–defense trees. In: Degano, P., Etalle, S., Guttman, J. (eds.) FAST 2010. LNCS, vol. 6561, pp. 80–95. Springer, Heidelberg (2011). https://doi.org/10.1007/978-3-642-19751-2_6
16. Larsen, M.E.V.: BCOol: the behavioral coordination operator language, Ph. D. thesis, University of Nice Sophia Antipolis, France (2016)
17. Larsen, M.E.V., DeAntoni, J., Combemale, B., Mallet, F.: A behavioral coordination operator language (BCOoL). In: Proceedings of the 18th International Conference on Model Driven Engineering Languages and Systems. MODELS 2015, IEEE Press (2015)
18. Leuschel, M., Butler, M.: ProB: an automated analysis toolset for the B method. Int. J. Softw. Tools Technol. Transfer **10**, 185–203 (2008)
19. Milner, R.: A Calculus of Communicating Systems. Springer-Verlag, Berlin, Heidelberg (1982). https://doi.org/10.1007/3-540-10235-3
20. OMG: Business Process Model and Notation (BPMN), Version 2.0 (2011). http://www.omg.org/spec/BPMN/2.0
21. OMG: Unified modeling language^{TM} (uml®) (2011). http://www.omg.org/spec/UML/index.htm
22. Roscoe, A.W.: The Theory and Practice of Concurrency. Prentice Hall PTR, USA (1997)
23. Schneider, S., Treharne, H.: CSP Theorems for Communicating B Machines. Form. Asp. Comput. **17**(4), 390–422 (2005). https://doi.org/10.1007/s00165-005-0076-7
24. Tourchi Moghaddam, M., Rutten, E., Giraud, G.: Hierarchical control for self-adaptive IoT systems : a constraint programming-based adaptation approach. In: HICSS 2022 - Hawaii International Conference on System Sciences, pp. 1–10. Hawaii, United States (2022). http://hal.inria.fr/hal-03461137